



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №7

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 Сервисы	3
1.2 Диалоговые окна	8
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	17
2.1 Создание и запуск сервиса	17
2.2 Создание различных диалоговых окон	18
2.2.1 AlertDialog	18
2.2.2 DatePickerDialog	21
2.2.3 TimePickerDialog	21
2.2.4 Custom Dialog.....	22
2.2.5 Итоговый вид приложения.....	23
ЗАКЛЮЧЕНИЕ	30

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 Сервисы

Сервис в Android — это компонент приложения, предназначенный для выполнения длительных или ресурсоемких операций в фоновом режиме без предоставления пользовательского интерфейса. Сервисы могут работать в фоне даже тогда, когда пользователь не взаимодействует с приложением, что делает их идеальными для таких задач, как воспроизведение музыки, выполнение сетевых операций, выполнение вычислений и мониторинг данных.

Все сервисы наследуются от класса `Service` и по аналогии с `activity`, сервис имеет свой жизненный цикл и методы (рисунок 1.1.1):

- `onCreate()`;
- `onStartCommand(Intent intent, int flags, int startId)`;
- `onBind(Intent intent)`;
- `onUnbind(Intent intent)`;
- `onDestroy()`.

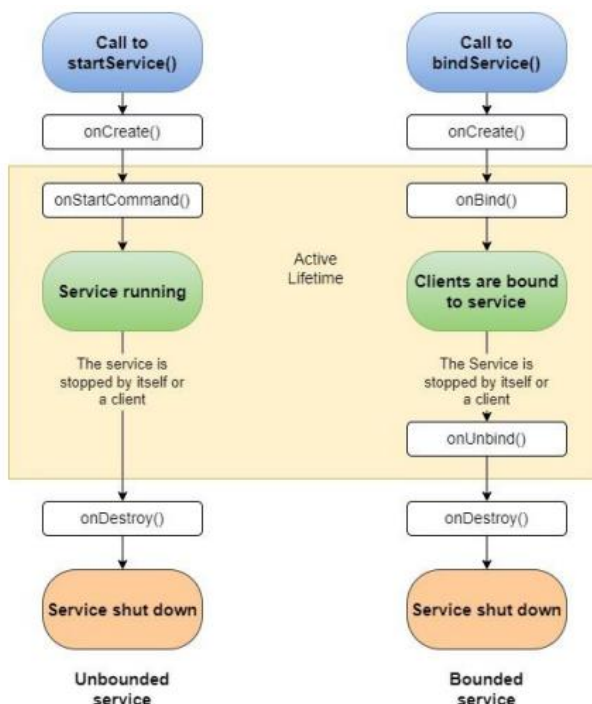


Рисунок 1.1.1 – Жизненный цикл сервиса

Метод `onCreate()` вызывается при создании сервиса. Это первый вызов, который получает сервис, и он используется для однократной инициализации, такой как создание ресурсов, которые будут использоваться в течение всего времени существования сервиса. Метод `onCreate()` вызывается только один раз перед вызовом `onStartCommand()` или `onBind()`.

Метод `onStartCommand()` вызывается каждый раз, когда компонент (например, активность) запрашивает запуск сервиса через `startService()`. В этом методе сервис может выполнять любые операции, включая запуск потока для выполнения сложной задачи в фоне. Метод возвращает константу, указывающую, как система должна вести себя, если сервис уничтожается до того, как он завершит выполнение своей работы.

Метод `onBind()` вызывается, когда другой компонент хочет привязаться к сервису через `bindService()`. Если сервис не предоставляет интерфейс для клиентов, то он должен возвращать `null`.

Метод `onUnbind()` вызывается, когда все клиенты отсоединились от определенного интерфейса сервиса. После этого вызова, если необходимо, сервис может остановить себя через `stopSelf()`.

Метод `onDestroy()` вызывается, когда сервис больше не используется и собирается быть уничтоженным. Это последний вызов, который получает сервис, и он используется для освобождения ресурсов, таких как потоки, зарегистрированные приемники, обработчики и т.д.

В качестве примера, создадим сервис, который будет воспроизводить музыку. Предварительно загрузим медиафайл формата `mp3`.

Чтобы это сделать необходимо создать в папке `res` папку `raw`. Именно в этой папке хранятся различные файлы, которые сохраняются в исходном виде. После этого нужно загрузить медиафайл в папку `res/raw` таким же способом, как добавляем изображения.

После того, как файл добавлен нужно создать новый класс сервиса. Это можно сделать, нажав правой кнопкой мыши на "java" → "New" → "Service" → "Service" (рисунок 1.1.2).

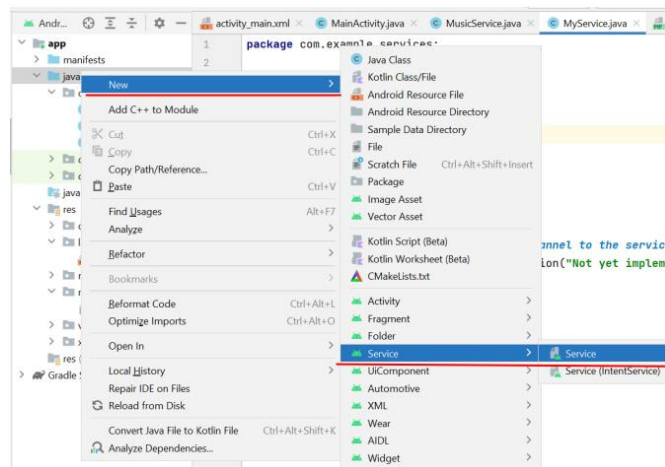


Рисунок 1.1.2 – Создание класса сервиса

Теперь нужно прописать логику работы сервиса (рисунок 1.1.3).

```
<? activity_main.xml MainActivity.java MusicService.java
1 package com.example.prac7;
2
3 import ...
4
5 public class MusicService extends Service {
6     2 usages
7     private static final String TAG = "MusicService";
8     8 usages
9     private MediaPlayer mediaPlayer;
10    @Override
11    public void onCreate() {
12        super.onCreate();
13        // Инициализация медиаплеера
14        mediaPlayer = MediaPlayer.create(this, R.raw.music);
15        mediaPlayer.setLooping(true); // За цикливание воспроизведения
16        mediaPlayer.setVolume(100, 100);
17    }
18    @Override
19    public int onStartCommand(Intent intent, int flags, int startId) {
20        if (!mediaPlayer.isPlaying()) {
21            mediaPlayer.start();
22            Log.d(TAG, "Музыка начала играть");
23        }
24        return START_STICKY;
25    }
26    @Override
27    public void onDestroy() {
28        super.onDestroy();
29        if (mediaPlayer.isPlaying()) {
30            mediaPlayer.stop();
31            mediaPlayer.release();
32            Log.d(TAG, "Музыка остановлена и ресурсы освобождены");
33        }
34    }
35    @Override
36    public IBinder onBind(Intent intent) {
37        return null; // Для сервисов без привязки возвращаем null
38    }
39 }
40
41
42 }
```

Рисунок 1.1.3 – Описание логики работы класса-сервиса

Для воспроизведения музыкального файла сервис будет использовать компонент MediaPlayer.

В сервисе переопределяются все четыре метода жизненного цикла. Но по сути метод onBind() не имеет никакой реализации.

В методе onCreate() инициализируется медиа-проигрыватель с помощью музыкального ресурса, который добавлен в папку res/raw.

В методе onStartCommand() начинается воспроизведение.

Метод onStartCommand() может возвращать одно из значений, которое предполагает различное поведение в случае, если процесс сервиса был неожиданно завершён системой:

- START_STICKY: в этом случае сервис снова возвращается в запущенное состояние, как будто если бы снова был бы вызван метод onStartCommand() без передачи в этот метод объекта Intent.
- START_REDELIVER_INTENT: в этом случае сервис снова возвращается в запущенное состояние, как будто если бы снова был бы вызван метод onStartCommand() с передачей в этот метод объекта Intent.
- START_NOT_STICKY: сервис остается в остановленном положении. Метод onDestroy() завершает воспроизведение.

Далее, объявим сервис в файле AndroidManifest.xml. При создании сервиса изменения в файле манифеста произойдут автоматически, но если этого не произошло, то необходимо прописать вручную объявление сервиса (рисунок 1.1.4).



```
<application
    android:allowBackup="true"
    android:dataExtractionRules="@xml/data_extraction_rules"
    android:fullBackupContent="@xml/backup_rules"
    android:icon="@mipmap/ic_launcher"
    android:label="Services"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportsRtl="true"
    android:theme="@style/Theme.Services"
    tools:targetApi="31" >
    <service
        android:name=".MusicService"
        android:enabled="true"
        android:exported="true" />
    <activity
        android:name=".MainActivity"
```

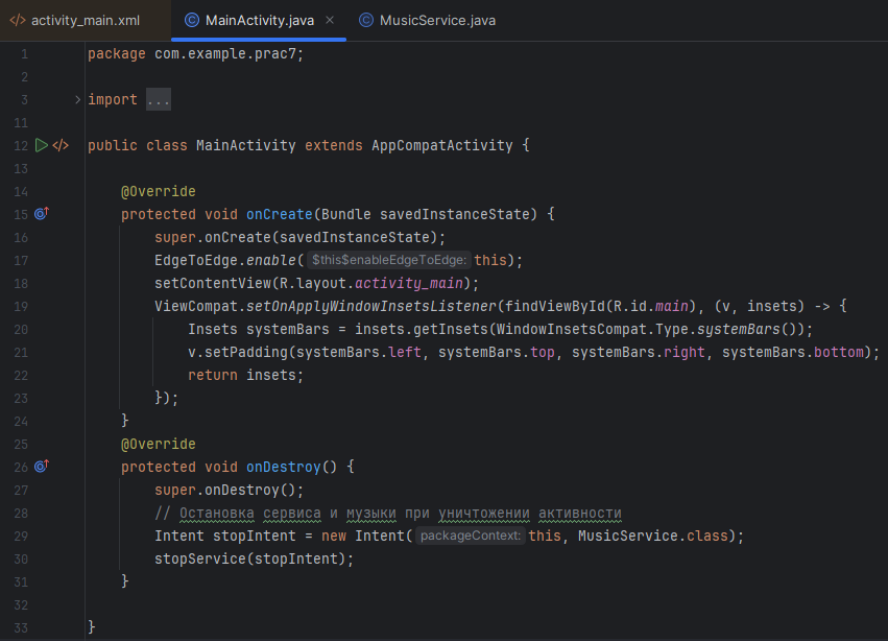
Рисунок 1.1.4 – Объявление сервиса

Регистрация сервиса производится в узле application с помощью добавления элемента . В нем определяется атрибут android:name, который

хранит название класса сервиса. И, кроме того, может принимать еще ряд атрибутов:

- android:enabled: если имеет значение "true", то сервис может ли создаваться системой. Значение по умолчанию - "true";
- android:exported: указывает, могут ли компоненты других приложений обращаться к сервису. Если имеет значение "true", то могут, если имеет значение "false", то нет;
- android:icon: значок сервиса, представляет собой ссылку на ресурс drawable;
- android:isolatedProcess: если имеет значение true, то сервис может быть запущен как специальный процесс, изолированный от остальной системы;
- android:label: название сервиса, которое отображается пользователю;
- android:permission: набор разрешений, которые должно применять приложение для запуска сервиса;
- android:process: название процесса, в котором запущен сервис. Как правило, имеет то же название, что и пакет приложения;

Теперь можно управлять сервисом (запускать и останавливать воспроизведение) из активности (рисунок 1.1.5).



```
1 package com.example.prac7;
2
3 > import ...
11
12 public class MainActivity extends AppCompatActivity {
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17         EdgeToEdge.enable(this);
18         setContentView(R.layout.activity_main);
19         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
20             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
21             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
22             return insets;
23         });
24     }
25
26     @Override
27     protected void onDestroy() {
28         super.onDestroy();
29         // Остановка сервиса и музыки при уничтожении активности
30         Intent stopIntent = new Intent(packageContext, this, MusicService.class);
31         stopService(stopIntent);
32     }
33 }
```

Рисунок 1.1.5 – Описание логики управления сервисом

Для запуска сервиса в классе Activity определен метод `startService()`, в который передается объект `Intent`. Этот метод будет посылать команду сервису и вызывать его метод `onStartCommand()`, а также указывать системе, что сервис должен продолжать работать до тех пор, пока не будет вызван метод `stopService()`.

Метод `stopService()` также определен в классе Activity и принимает объект `Intent`. Он останавливает работу сервиса, вызывая его метод `onDestroy()`.

Теперь если запустить созданный проект, то начнет играть музыка, при этом на экране ничего не будет отображаться, только если мы сами не зададим.

1.2 Диалоговые окна

Диалоговые окна в Android представляют собой небольшие окна, которые отображаются поверх основного содержимого приложения для передачи важного сообщения пользователю или запроса действия от пользователя. Эти окна могут использоваться для подтверждения действий, информирования о событиях, ввода данных и других задач, требующих внимания пользователя.

Среди основных типов диалоговых окон можно выделить:

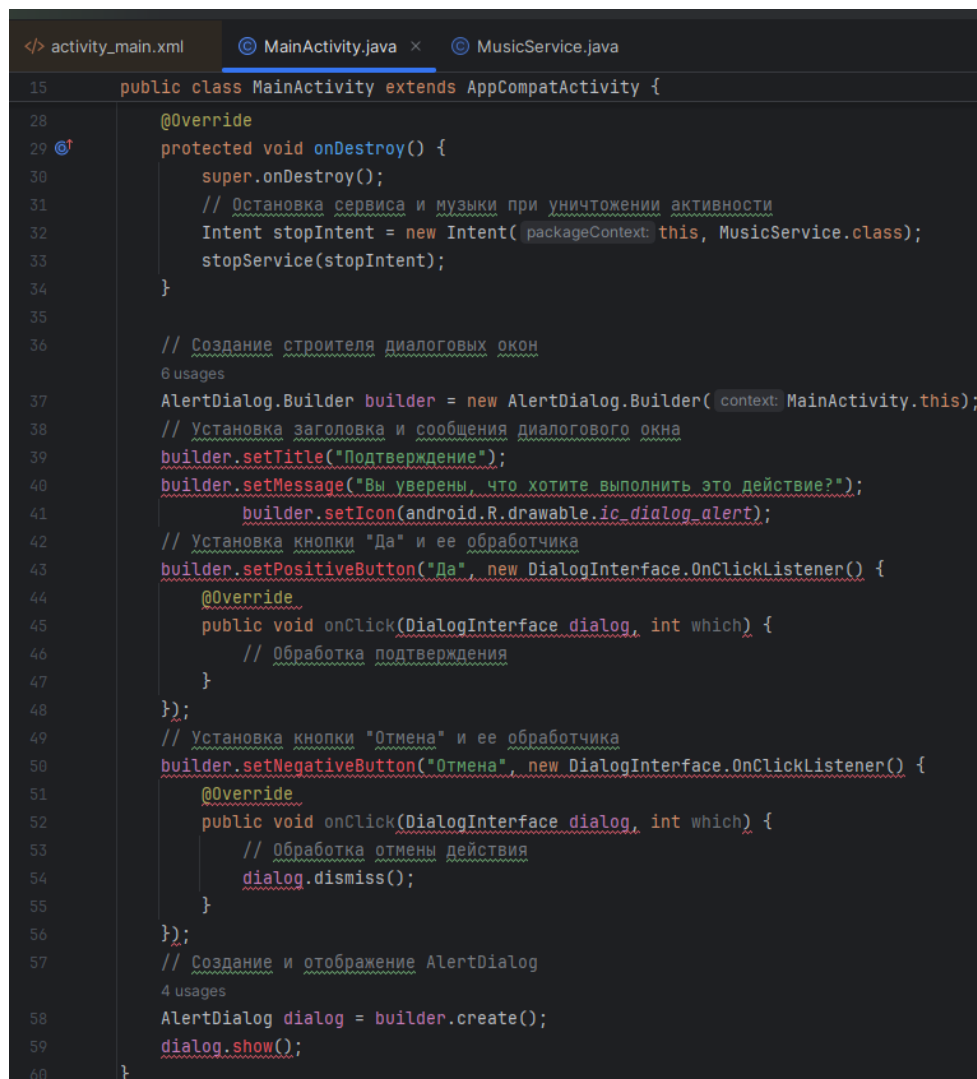
- . `AlertDialog`: используется для отображения предупреждений и предложения пользователю совершить выбор. Может включать кнопки для обработки действий пользователя, такие как «Да», «Нет» или «Отмена»;
- `DatePickerDialog` и `TimePickerDialog`: позволяют пользователю выбрать дату или время соответственно;
- `Custom Dialog`: Позволяет создавать диалоговые окна с пользовательским макетом, что может включать любые элементы управления;

В создаваемых диалоговых окнах можно задавать следующие элементы:

- заголовок;
- текстовое сообщение;
- кнопки: от одной до трёх;
- список;
- флажки;
- переключатели.

Разберем созданием каждого типа диалоговых окон.

Начнём с AlertDialog с простого примера – покажем на экране диалоговое окно с вопросом и парой кнопок. Для этого в файле MainActivity пропишем код, реализующий эту возможность.



```
<? activity_main.xml    MainActivity.java    MusicService.java
15    public class MainActivity extends AppCompatActivity {
28        @Override
29        protected void onDestroy() {
30            super.onDestroy();
31            // Остановка сервиса и музыки при уничтожении активности
32            Intent stopIntent = new Intent(packageContext: this, MusicService.class);
33            stopService(stopIntent);
34        }
35
36        // Создание строителя диалоговых окон
37        6 usages
38        AlertDialog.Builder builder = new AlertDialog.Builder(context: MainActivity.this);
39        // Установка заголовка и сообщения диалогового окна
40        builder.setTitle("Подтверждение");
41        builder.setMessage("Вы уверены, что хотите выполнить это действие?");
42        builder.setIcon(android.R.drawable.ic_dialog_alert);
43        // Установка кнопки "Да" и ее обработчика
44        builder.setPositiveButton("Да", new DialogInterface.OnClickListener() {
45            @Override
46            public void onClick(DialogInterface dialog, int which) {
47                // Обработка подтверждения
48            }
49        });
50        // Установка кнопки "Отмена" и ее обработчика
51        builder.setNegativeButton("Отмена", new DialogInterface.OnClickListener() {
52            @Override
53            public void onClick(DialogInterface dialog, int which) {
54                // Обработка отмены действия
55                dialog.dismiss();
56            }
57        });
58        // Создание и отображение AlertDialog
59        4 usages
60        AlertDialog dialog = builder.create();
61        dialog.show();
62    }
```

Рисунок 1.2.1 – Реализация диалогового окна

Таким образом будет создано диалоговое окно с кнопками «Да» и «Отмена», которое может обрабатывать действия пользователя (рисунок 1.2.2).

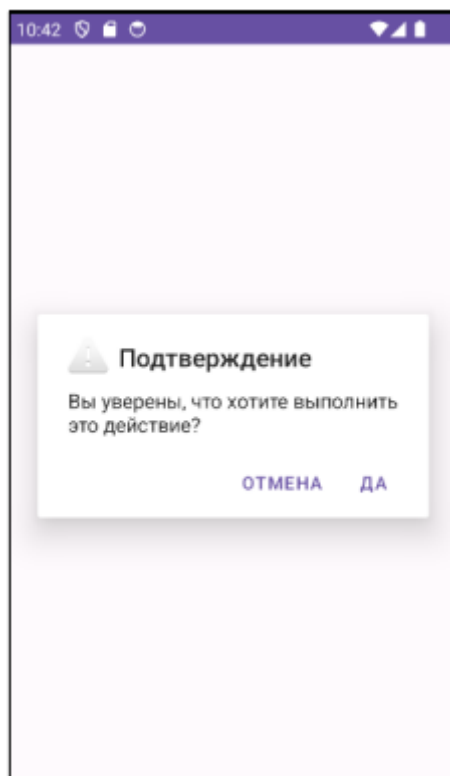


Рисунок 1.2.2 – Отображение итогового вида приложения

Класс `AlertDialog.Builder` с помощью своих методов позволяет настроить отображение диалогового окна:

- `setTitle`: устанавливает заголовок окна;
- `setView`: устанавливает разметку интерфейса окна;
- `setIcon`: устанавливает иконку окна;
- `setPositiveButton`: устанавливает кнопку подтверждения действия;
- `setNeutralButton`: устанавливает "нейтральную" кнопку, действие которой может отличаться от действий подтверждения или отмены;
- `setNegativeButton`: устанавливает кнопку отмены;
- `setMessage`: устанавливает текст диалогового окна, но при использовании `setView` данный метод необязателен или может рассматриваться в качестве альтернативы, если нам надо просто вывести сообщение;
- `create`: создает окно.

Аналогичным образом создаются диалоговые окна с выбором времени или даты (рисунки 1.2.3-1.2.4).

```
1 usage
private int hour;
1 usage
private int minute; } Не забыть создать переменные
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    // Создание и отображение TimePickerDialog
    TimePickerDialog timePickerDialog = new TimePickerDialog(
        context: this,
        new TimePickerDialog.OnTimeSetListener() {
            @Override
            public void onTimeSet(TimePicker view, int hourOfDay, int minute) {
                // Обработка выбранного времени
                // Пример: установка времени в TextView
                // textView.setText(hourOfDay + ":" + minute);
            }
        }, hour, minute, is24HourView: true); // Использование 24-часового формата
    timePickerDialog.show();
}
```

Рисунок 1.2.3 – Реализация логики диалогового окна с выбором времени



Рисунок 1.2.4 - Отображение итогового вида приложения

Далее идёт создание диалогового окна с выбором даты (рисунки 1.2.5-1.2.6).

```

private int year;
1 usage
private int month;
1 usage
private int day;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    // Создание обработчика выбора даты
    DatePickerDialog.OnDateSetListener dateSetListener = new DatePickerDialog.OnDateSetListener() {
        @Override
        public void onDateSet(DatePicker view, int year, int month, int dayOfMonth) {
            // Обработка выбора даты
        }
    };
    // Создание и отображение DatePickerDialog
    DatePickerDialog datePickerDialog = new DatePickerDialog(
        context: MainActivity.this,
        dateSetListener,
        year, // текущий год
        month, // текущий месяц
        day); // текущий день
    datePickerDialog.show();
}

```

Не забываем создать переменные

Рисунок 1.2.5 - Реализация логики диалогового окна с выбором даты



Рисунок 1.2.6 - Отображение итогового вида приложения

Создание пользовательского диалогового окна отличается установкой макета того, как будет выглядеть окно и какие элементы пользовательского интерфейса будет иметь. Для этого необходимо создать отдельный макет, например `custom_dialog` и наполнить его элементами пользовательского

интерфейса. Например, добавим текстовой поле с вопросом и две кнопки для выбора (рисунки 1.2.7-1.2.8).

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        // Создание диалога
        Dialog dialog = new Dialog( context: MainActivity.this);
        // Установка макета для диалогового окна
        dialog.setContentView(R.layout.custom_dialog);
        // Настройка элементов в макете
        Button button = dialog.findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                // Обработка нажатия на кнопку
            }
        });
        // Отображение диалогового окна
        dialog.show();
    }
}
```

Рисунок 1.2.7 – Реализация логики пользовательского диалогового окна

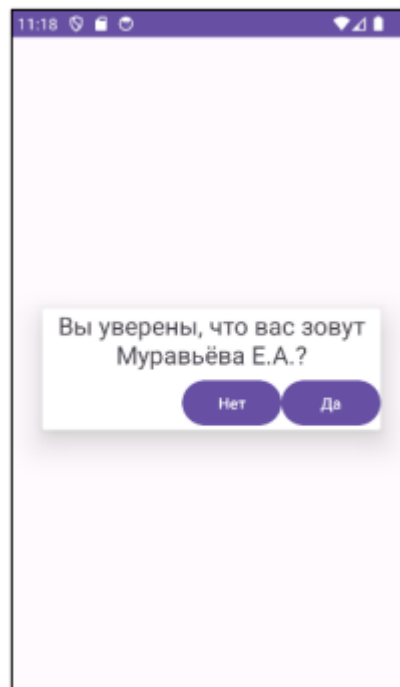


Рисунок 1.2.8 - Отображение итогового вида приложения

Кроме того, что диалоговое окно можно вызвать напрямую в MainActivity, его можно выделить в отдельный класс фрагмента DialogFragment.

Использование фрагментов для диалоговых окон в силу своей архитектуры является удобным вариантом в приложениях, который лучше справляется с поворотами устройства, нажатием кнопки "Назад", лучше подходит под разные размеры экранов и т.д.

Для создания диалога следует наследоваться от класса `DialogFragment`. Вначале добавляем в проект новый класс фрагмента, который назовем `MyDialogFragment`.

Класс фрагмента содержит всю стандартную функциональность фрагмента с его жизненным циклом, но при этом наследуется от класса `DialogFragment`, который добавляет ряд дополнительных функций. И для его создания мы можем использовать два способа:

- переопределение метода `onCreateDialog()`, который возвращает объект `Dialog`;
- использование стандартного метода `onCreateView()`.

Для создания диалогового окна в методе `onCreateDialog()` применяется класс `AlertDialog.Builder`, также как и в `MainActivity` (рисунок 1.2.9).

```
public class MyDialogFragment extends DialogFragment {  
    3 usages  
    @NonNull  
    @Override  
    public Dialog onCreateDialog(Bundle savedInstanceState) {  
  
        AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());  
        // Установка заголовка, сообщения, иконки диалогового окна  
        builder.setTitle("Подтверждение");  
        builder.setMessage("Вы уверены, что хотите выполнить это действие?");  
        builder.setIcon(android.R.drawable.ic_dialog_alert);  
        builder.setNegativeButton( text: "Отмена", listener: null);  
        builder.setPositiveButton( text: "Ок", listener: null);  
        return builder.create();  
    }  
}
```

Рисунок 1.2.9 – Наполнение класса `MyDialogFragment`

В данном случае для обработчика нажатия передается `null`, то есть обработчик не установлен.

Теперь диалоговое окно можно вызвать в классе `MainActivity`.

Для вызова диалогового окна создается объект фрагмента MyDialogFragment, затем у него вызывается метод show(). В этот метод передается менеджер фрагментов FragmentManager и строка – произвольный тег (рисунок 1.2.10).

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    MyDialogFragment dialogFragment = new MyDialogFragment();
    dialogFragment.show(getSupportFragmentManager(), tag: "Проверка");
}
```

Рисунок 1.2.10 - Конструктор экземпляров класса MyDialogFragment
Результат запуска будет таким же, как и ранее (рисунок 1.2.11).

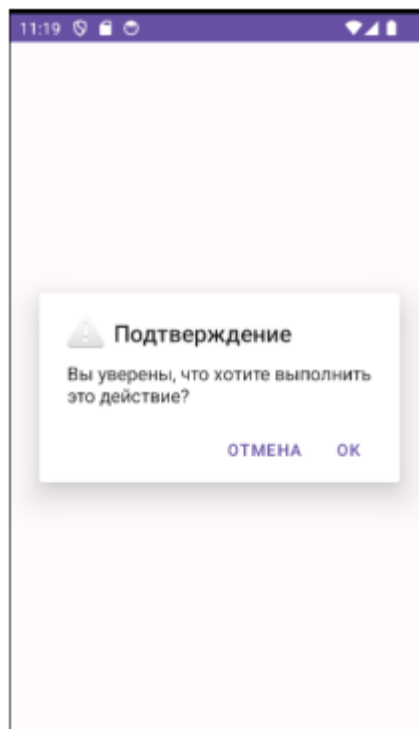


Рисунок 1.2.11 – Отображение итогового вида приложения

В случае, если в диалоговое окно необходимо передать данные для уточнения, то тогда в сообщении необходимо передавать текстовое значение.

Например, создадим поле для ввода фамилии и кнопку для обработки нажатия. После нажатия на кнопку, у нас будет выводиться предупреждение (рисунки 1.2.12-1.2.13).


```

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    Button alertButton = findViewById(R.id.buttonAlert);
    AlertDialog.Builder builder = new AlertDialog.Builder(context: MainActivity.this);
    alertButton.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            EditText nameText = findViewById(R.id.textName);
            String name = nameText.getText().toString();
            builder.setTitle("Подтверждение");
            builder.setMessage("Вы уверены, что хотите удалить " + name + " ?");
            builder.create().show();
        }
    });
}

```

Получает введенный текст

Передаем текст в Alert

Рисунок 1.2.12 – Описание логики файла разметки с Button и EditText

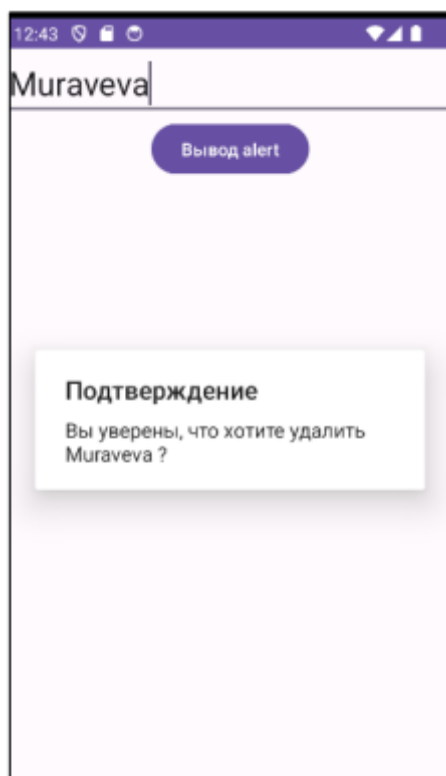


Рисунок 1.2.13 – Итоговый вид приложения

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Создание и запуск сервиса

Для создания сервиса для воспроизведения музыки необходимо добавить файл в соответствующим содержимым в папку `resources/raw` (рисунок 2.1.1).

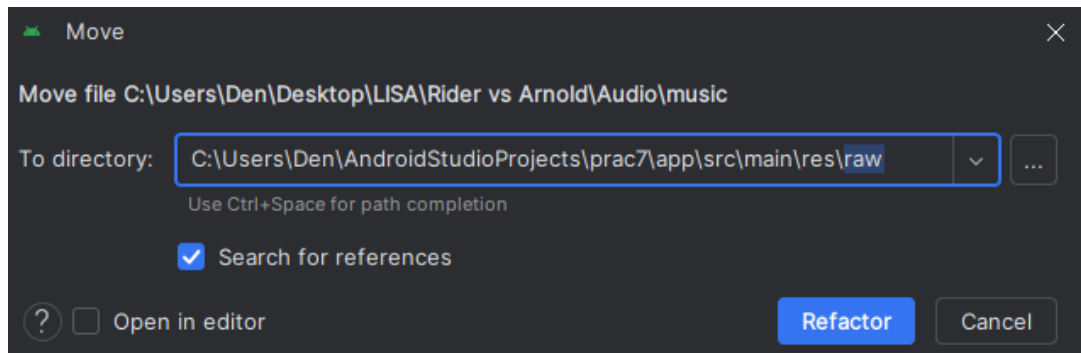


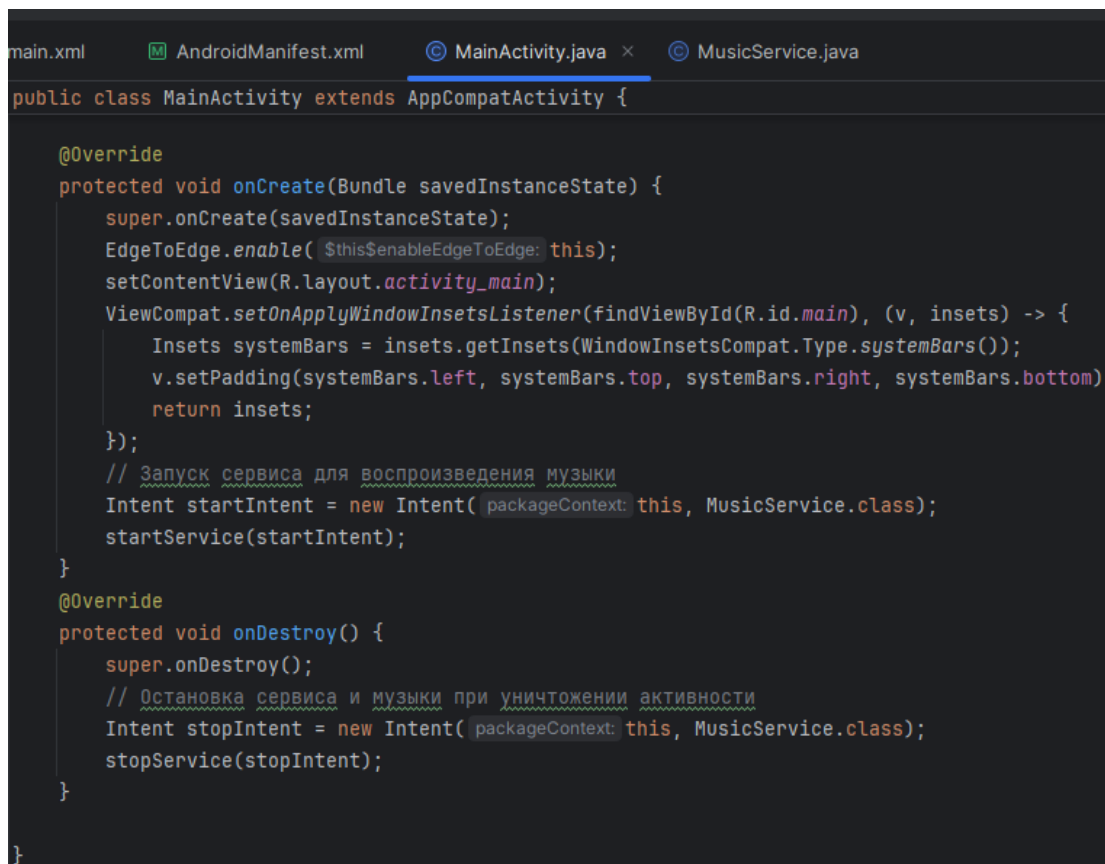
Рисунок 2.1.1 – Добавление звукового ресурса

Далее следует описание класса сервиса, которое включает в себя логику воспроизведения музыки: начало воспроизведения и его завершение (рисунок 2.1.2).

```
<> activity_main.xml  AndroidManifest.xml  MainActivity.java  MusicService.java x
1  package com.example.prac7;
2
3  > import ...
4
5  </>
6  public class MusicService extends Service {
7      no usages
8      public MusicService() {}
9      8 usages
10     private MediaPlayer mediaPlayer;
11     @Override
12     public void onCreate() {
13         super.onCreate();
14         // Инициализация медиаплеера
15         mediaPlayer = MediaPlayer.create(context, this, R.raw.music);
16         mediaPlayer.setLooping(true); // Зацикливание воспроизведения
17         mediaPlayer.setVolume(leftVolume: 100, rightVolume: 100);
18     }
19
20     @Override
21     public int onStartCommand(Intent intent, int flags, int
22         startId) {
23         if (!mediaPlayer.isPlaying()) {
24             mediaPlayer.start();
25         }
26         return START_STICKY;
27     }
28
29     @Override
30     public void onDestroy() {
31         super.onDestroy();
32         if (mediaPlayer.isPlaying()) {
33             mediaPlayer.stop();
34             mediaPlayer.release();
35         }
36     }
37
38     @Override
39     public IBinder onBind(Intent intent) {
40         return null; // Для сервисов без привязки возвращаем null
41     }
42 }
```

Рисунок 2.1.2 – Часть 1 описания логики класса сервиса воспроизведения музыки

Также необходимо определить запуск и окончание работы сервиса в выбранной activity (MainActivity.java) (рисунок 2.1.3).



```
main.xml  AndroidManifest.xml  MainActivity.java  MusicService.java

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_main);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        // Запуск сервиса для воспроизведения музыки
        Intent startIntent = new Intent( packageContext: this, MusicService.class);
        startService(startIntent);
    }

    @Override
    protected void onDestroy() {
        super.onDestroy();
        // Остановка сервиса и музыки при уничтожении активности
        Intent stopIntent = new Intent( packageContext: this, MusicService.class);
        stopService(stopIntent);
    }

}
```

Рисунок 2.1.3 – Описание логики сервиса воспроизведения музыки в activity

При запуске приложения будет зациклено воспроизводиться звуковой файл.

2.2 Создание различных диалоговых окон

2.2.1 AlertDialog

Для начала работы с диалоговыми окнами необходимо создать класс фрагментов для дальнейшего отображения результатов ввода пользователя (рисунок 2.2.1.1-2.2.1.2).

```

main.xml  DialogFragment.java  AndroidManifest.xml  MainActivity.java

public class DialogFragment extends Fragment {

    // TODO: Rename parameter arguments, choose names that match
    // the fragment initialization parameters, e.g. ARG_ITEM_NUMBER
    2 usages
    private static final String ARG_PARAM1 = "param1";
    2 usages
    private static final String ARG_PARAM2 = "param2";

    // TODO: Rename and change types of parameters
    1 usage
    private String mParam1;
    1 usage
    private String mParam2;

    3 usages
    private TextView textView;

    public DialogFragment() {
        // Required empty public constructor
    }

    /**
     * Use this factory method to create a new instance of
     * this fragment using the provided parameters.
     *
     * @param param1 Parameter 1.
     * @param param2 Parameter 2.
     * @return A new instance of fragment DialogFragment.
     */
    // TODO: Rename and change types and number of parameters
    1 usage
    public static DialogFragment newInstance(String param1, String param2) {
        DialogFragment fragment = new DialogFragment();
        Bundle args = new Bundle();
        args.putString(ARG_PARAM1, param1);
        args.putString(ARG_PARAM2, param2);
        fragment.setArguments(args);

        return fragment;
    }
}

```

Рисунок 2.2.1.1 – Часть 1 описания логики класса DialogFragment

На рисунке 2.2.1.1 в базовый поля класса DialogFragment было добавлено поле textView, которое необходимо для отображения данных, введённых пользователем.

```

activity_main.xml  DialogFragment.java  AndroidManifest.xml  MainActivity.java  activity_alert.xml

public class DialogFragment extends Fragment {

    // TODO: Rename and change types and number of parameters
    1 usage
    @
    public static DialogFragment newInstance(String param1, String param2) {
        DialogFragment fragment = new DialogFragment();
        Bundle args = new Bundle();
        args.putString(ARG_PARAM1, param1);
        args.putString(ARG_PARAM2, param2);
        fragment.setArguments(args);

        return fragment;
    }

    @Override
    6
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (getArguments() != null) {
            mParam1 = getArguments().getString(ARG_PARAM1);
            mParam2 = getArguments().getString(ARG_PARAM2);
        }
    }

    @Override
    6 @
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.fragment_dialog, container, attachToRoot: false);
        TextView textView = (TextView) view.findViewById(R.id.text_view);
        return view;
    }

    5 usages
    public void updateText(String newText){
        if (textView != null)
            textView.setText(newText);
    }
}

```

Рисунок 2.2.1.2 - Часть 2 описания логики класса DialogFragment

На рисунке 2.2.1.2 отображено добавление метода `updateText` в дополнение к базовым методам. Данный метод необходим для внесения данных, введенных пользователем, в `textView`.

Опишем логику создания и работы диалогового окна `AlertDialog` (рисунки 2.2.1.3).

```

private void showAlertDialog() {
    AlertDialog.Builder builder = new AlertDialog.Builder(context: this);
    builder.setTitle("Подтверждение");
    builder.setMessage("Вы уверены, что хотите продолжить?");

    builder.setPositiveButton(text: "Да", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            DialogFragment fragment = (DialogFragment) getSupportFragmentManager().findFragmentById(R.id.alert_dialog);
            fragment.updateText(newText: "Вы ХОТИТЕ продолжить");
        }
    });

    builder.setNegativeButton(text: "Нет", new DialogInterface.OnClickListener() {
        @Override
        public void onClick(DialogInterface dialog, int which) {
            DialogFragment fragment = (DialogFragment) getSupportFragmentManager().findFragmentById(R.id.alert_dialog);
            fragment.updateText(newText: "Вы НЕ ХОТИТЕ продолжить");
        }
    });

    AlertDialog dialog = builder.create();
    dialog.show();
}

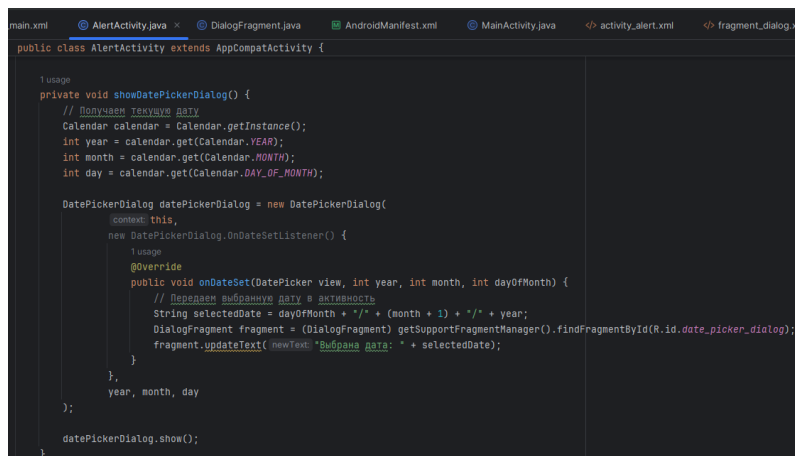
```

Рисунок 2.2.1.3 – Описание логики диалогового окна AlertDialog

На рисунке 2.2.1.3 описан функционал диалогового окна класса AlertDialog.

2.2.2 DatePickerDialog

Опишем создание диалогового окна класса DatePickerDialog (рисунок 2.2.2.1).



```
main.xml | AlertActivity.java | DialogFragment.java | AndroidManifest.xml | MainActivity.java | activity_alert.xml | fragment_dialog.xml
public class AlertActivity extends AppCompatActivity {

    1 usage
    private void showDatePickerDialog() {
        // Получаем текущую дату
        Calendar calendar = Calendar.getInstance();
        int year = calendar.get(Calendar.YEAR);
        int month = calendar.get(Calendar.MONTH);
        int day = calendar.get(Calendar.DAY_OF_MONTH);

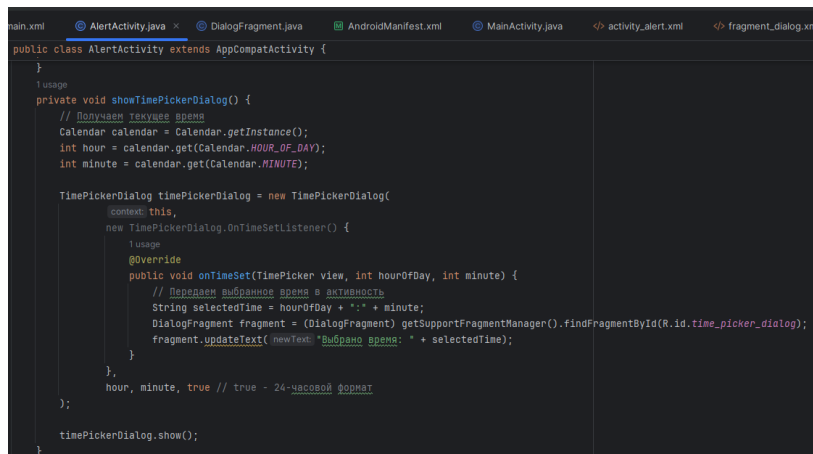
        DatePickerDialog datePickerDialog = new DatePickerDialog(
            context, this,
            new DatePickerDialog.OnDateSetListener() {
                1 usage
                @Override
                public void onDateSet(DatePicker view, int year, int month, int dayOfMonth) {
                    // Передаем выбранную дату в активность
                    String selectedDate = dayOfMonth + "/" + (month + 1) + "/" + year;
                    DialogFragment fragment = (DialogFragment) getSupportFragmentManager().findFragmentById(R.id.date_picker_dialog);
                    fragment.updateText(newText: "Выбрана дата: " + selectedDate);
                }
            },
            year, month, day
        );
        datePickerDialog.show();
    }
}
```

Рисунок 2.2.2.1 – Описание логики диалогового окна класса DatePickerDialog

На рисунке 2.2.2.1 отображён процесс сохранения даты, введённой пользователем, а также её отображение на TextView соответственного фрагмента.

2.2.3 TimePickerDialog

Опишем создание диалогового окна класса TimePickerDialog (рисунок 2.2.3.1).



```
main.xml | AlertActivity.java | DialogFragment.java | AndroidManifest.xml | MainActivity.java | activity_alert.xml | fragment_dialog.xml
public class AlertActivity extends AppCompatActivity {

    1 usage
    private void showTimePickerDialog() {
        // Получаем текущее время
        Calendar calendar = Calendar.getInstance();
        int hour = calendar.get(Calendar.HOUR_OF_DAY);
        int minute = calendar.get(Calendar.MINUTE);

        TimePickerDialog timePickerDialog = new TimePickerDialog(
            context, this,
            new TimePickerDialog.OnTimeSetListener() {
                1 usage
                @Override
                public void onTimeSet(TimePicker view, int hourOfDay, int minute) {
                    // Передаем выбранное время в активность
                    String selectedTime = hourOfDay + ":" + minute;
                    DialogFragment fragment = (DialogFragment) getSupportFragmentManager().findFragmentById(R.id.time_picker_dialog);
                    fragment.updateText(newText: "Выбрано время: " + selectedTime);
                }
            },
            hour, minute, true // true - 24-часовой формат
        );
        timePickerDialog.show();
    }
}
```

Рисунок 2.2.3.1 - Описание логики диалогового окна класса TimePickerDialog

На рисунке 2.2.3.1 отображён процесс сохранения времени, введённого пользователем, которое сохраняется в формате “hh:mm”, а также отображение времени на TextView соответственного фрагмента.

2.2.4 Custom Dialog

Для создания данного диалогового окна необходимо предварительно описать его файл разметки (рисунок 2.2.4.1).

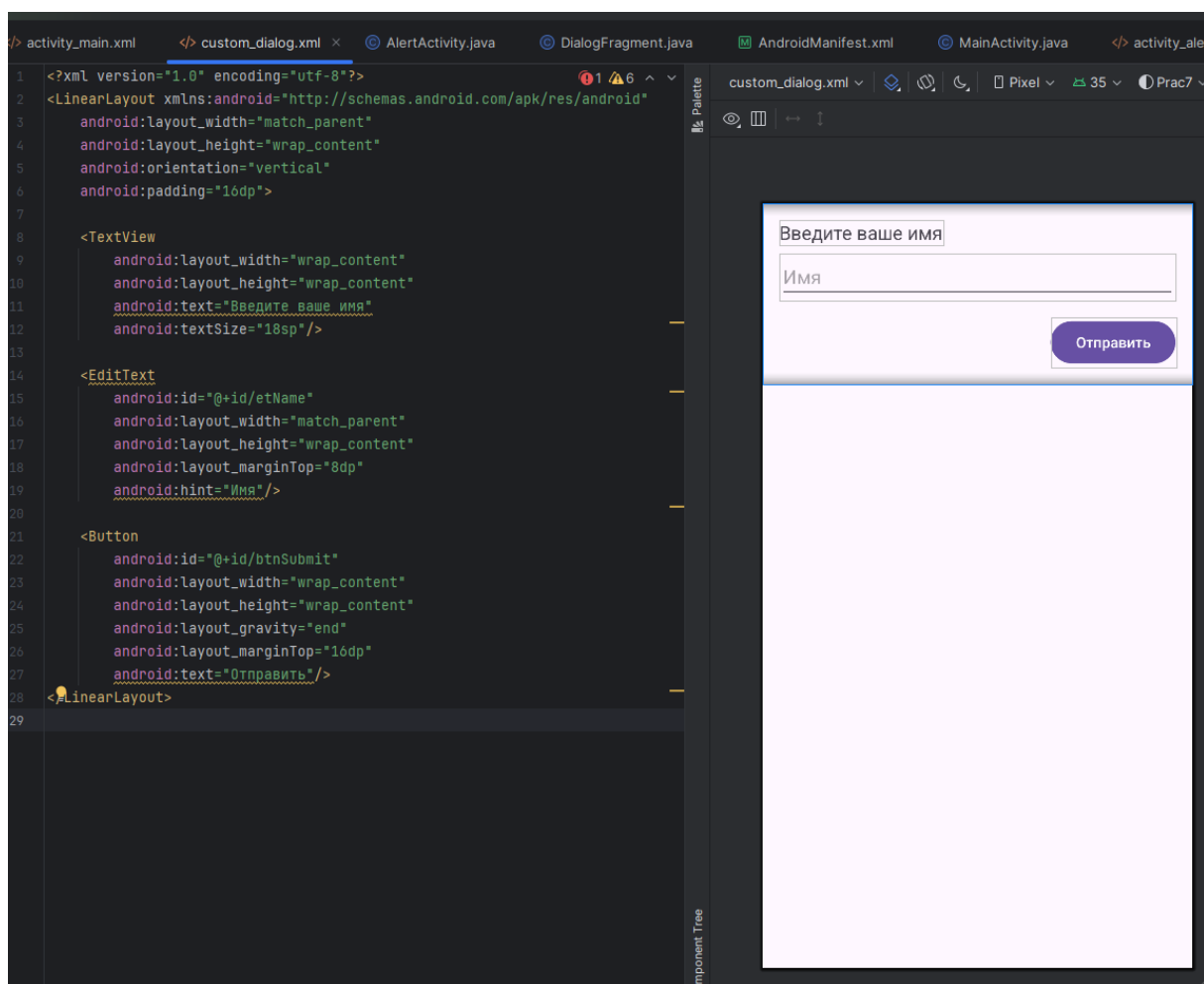


Рисунок 2.2.4.1 – Кодовый и графический вид файла разметки custom_dialog.xml

На рисунке 2.2.4.1 представлены элементы интерфейса, появляющиеся в пользовательском диалоговом окне: TextView (для отображения запроса), EditText (для ввода запрашиваемых данных), Button (для подтверждения введенных данных).

Опишем создание диалогового окна класса Dialog (рисунок 2.2.4.2).

```

public class AlertActivity extends AppCompatActivity {
    1 usage
    private void showCustomDialog() {
        final Dialog dialog = new Dialog(context: this);
        dialog setContentView(R.layout.custom_dialog);

        EditText etName = dialog.findViewById(R.id.etName);
        Button btnSubmit = dialog.findViewById(R.id.btnSubmit);

        btnSubmit.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                String name = etName.getText().toString();
                if (!name.isEmpty()) {
                    // Передаем данные в активность
                    DialogFragment fragment = (DialogFragment) getFragmentManager().findFragmentById(R.id.custom_dialog);
                    fragment.updateText(newText: "Вы ввели имя: " + name);
                    dialog.dismiss();
                } else {
                    etName.setError("Введите имя");
                }
            }
        });

        dialog.show();
    }
}

```

Рисунок 2.2.4.2 - Описание логики диалогового окна класса Dialog

На рисунке 2.2.4.2 отображена логика создания диалогового окна, а также обработка введённых данных.

2.2.5 Итоговый вид приложения

Теперь опишем активность, на которой отобразим созданные диалоговые окна (рисунки 2.2.5.1-2.2.5.3).

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_margin="25dp"
    tools:context=".AlertActivity">

    <Button
        android:id="@+id/btnAlert"
        android:text="Alert"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/btnDatePicker"
        android:text="DatePicker"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/btnTimePicker"
        android:text="TimePicker"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

    <Button
        android:id="@+id/btnCustom"
        android:text="Custom"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />


```

Рисунок 2.2.5.1 – Часть 1 графического и кодового вида файла разметки activity_alert.xml

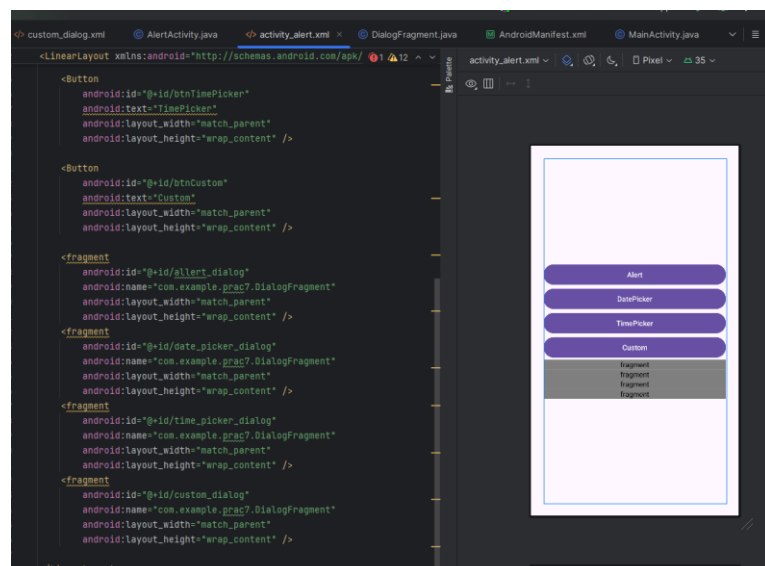


Рисунок 2.2.5.2 – Часть 2 графического и кодового вида файла разметки activity_alert.xml

На рисунках 2.2.5.1-2.2.5.2 последовательно показаны кнопки, вызывающие каждое из перечисленных выше диалоговых окон, а также фрагменты, соответственно отображающие данные, введённые пользователем.

Определим функционал файла разметки activity_alert.xml (2.2.5.3).

```

tom_dialog.xml  AlertActivity.java  activity_alert.xml  DialogFragment.java  AndroidManifest.x
public class AlertActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_alert);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        findViewById(R.id.btnAlert).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showAlertDialog();
            }
        });

        findViewById(R.id.btnDatePicker).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showDatePickerDialog();
            }
        });

        findViewById(R.id.btnTimePicker).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showTimePickerDialog();
            }
        });

        findViewById(R.id.btnCustom).setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                showCustomDialog();
            }
        });
    }
}

```

Рисунок 2.2.5.3 – Описание функционала файла разметки activity_alert.xml

На рисунке 2.2.5.3 отображено последовательное подключение ранее описанных методов к соответствующим кнопкам.

Далее представлен итоговый вид приложения (рисунки 2.2.5.4-2.2.5.8).

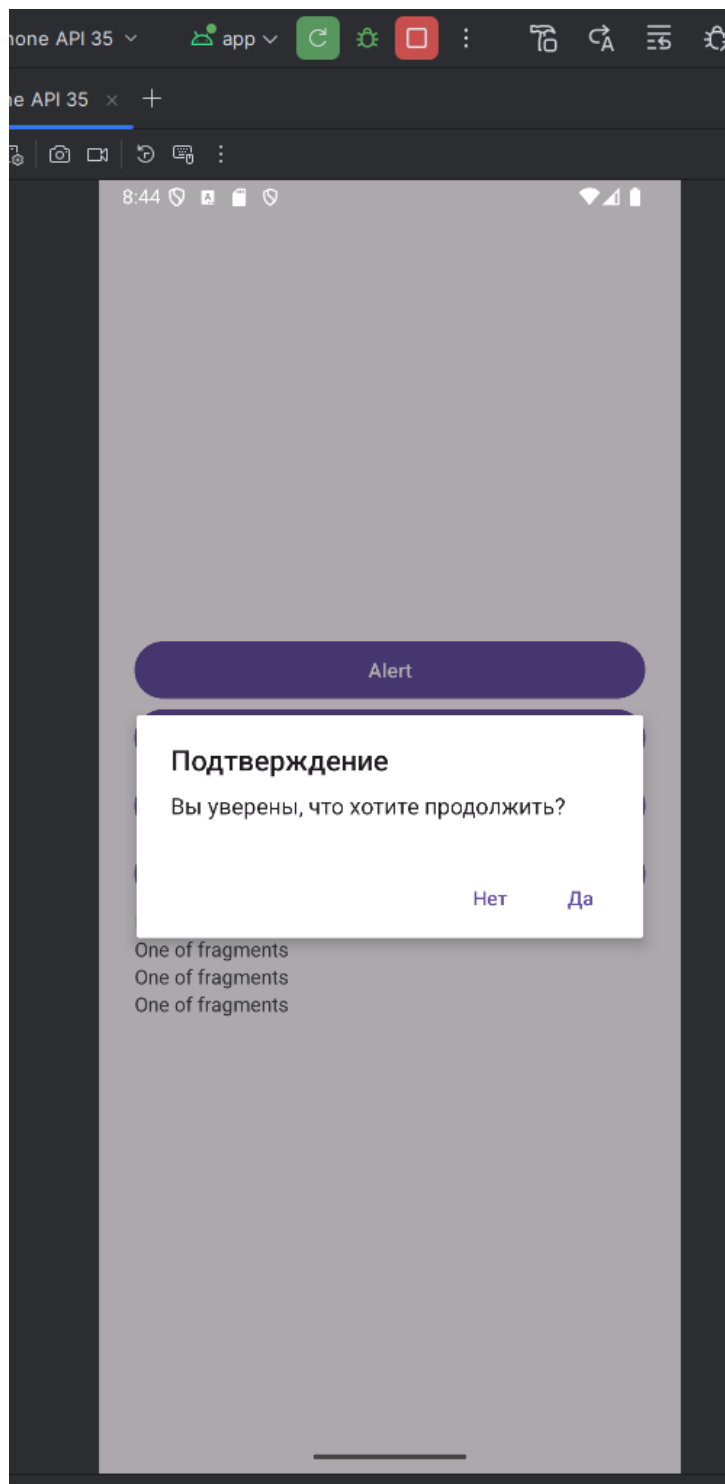


Рисунок 2.2.5.4 – Диалоговое окно класса AlertDialog

На рисунке 2.2.5.4 отображён выбор, предоставляемый пользователю при нажатии на кнопку с надписью Alert.

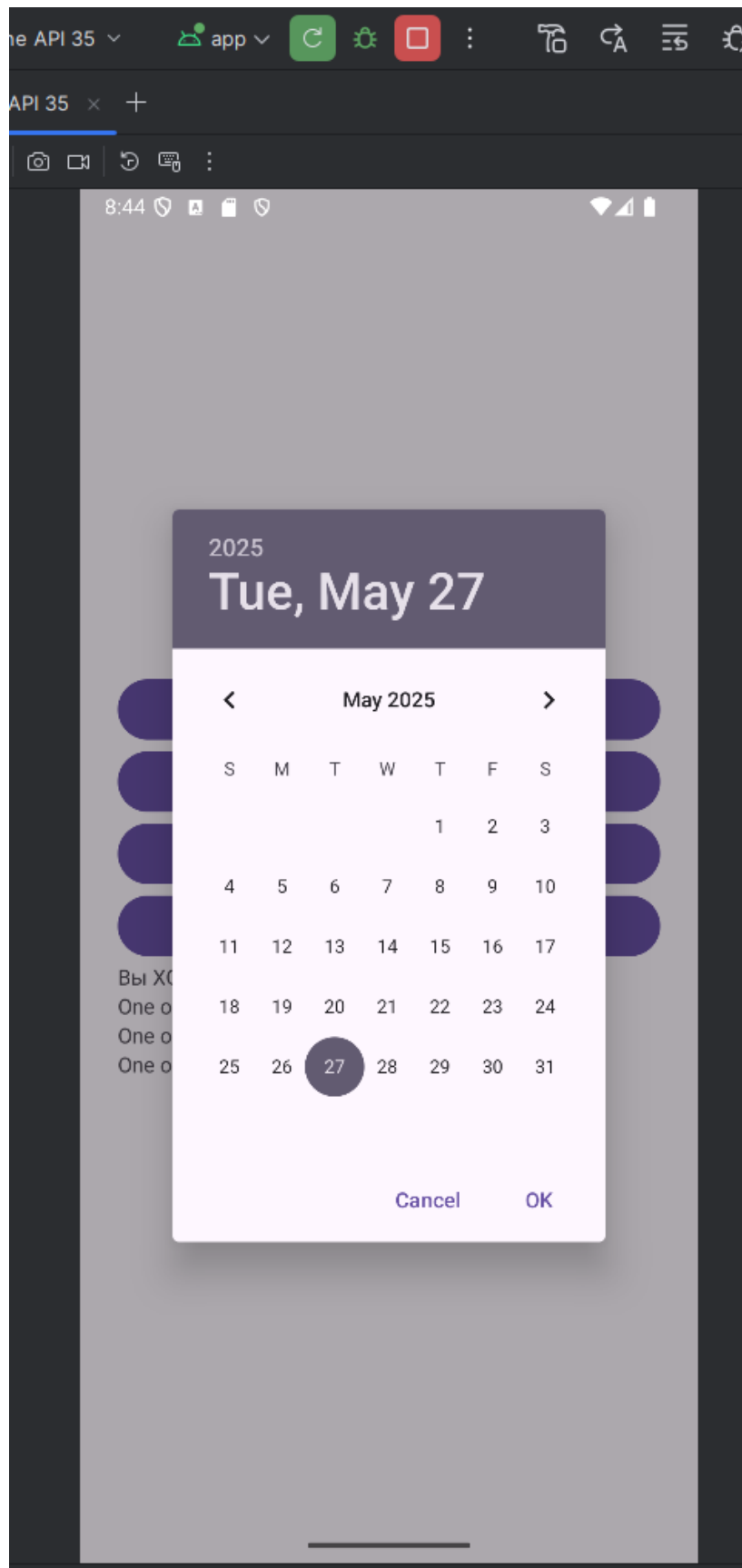


Рисунок 2.2.5.5 – Диалоговое окно класса DatePickerDialog

На рисунке 2.2.5.5 отображён выбор даты, предоставляемый пользователю при нажатии на кнопку с надписью DatePicker.

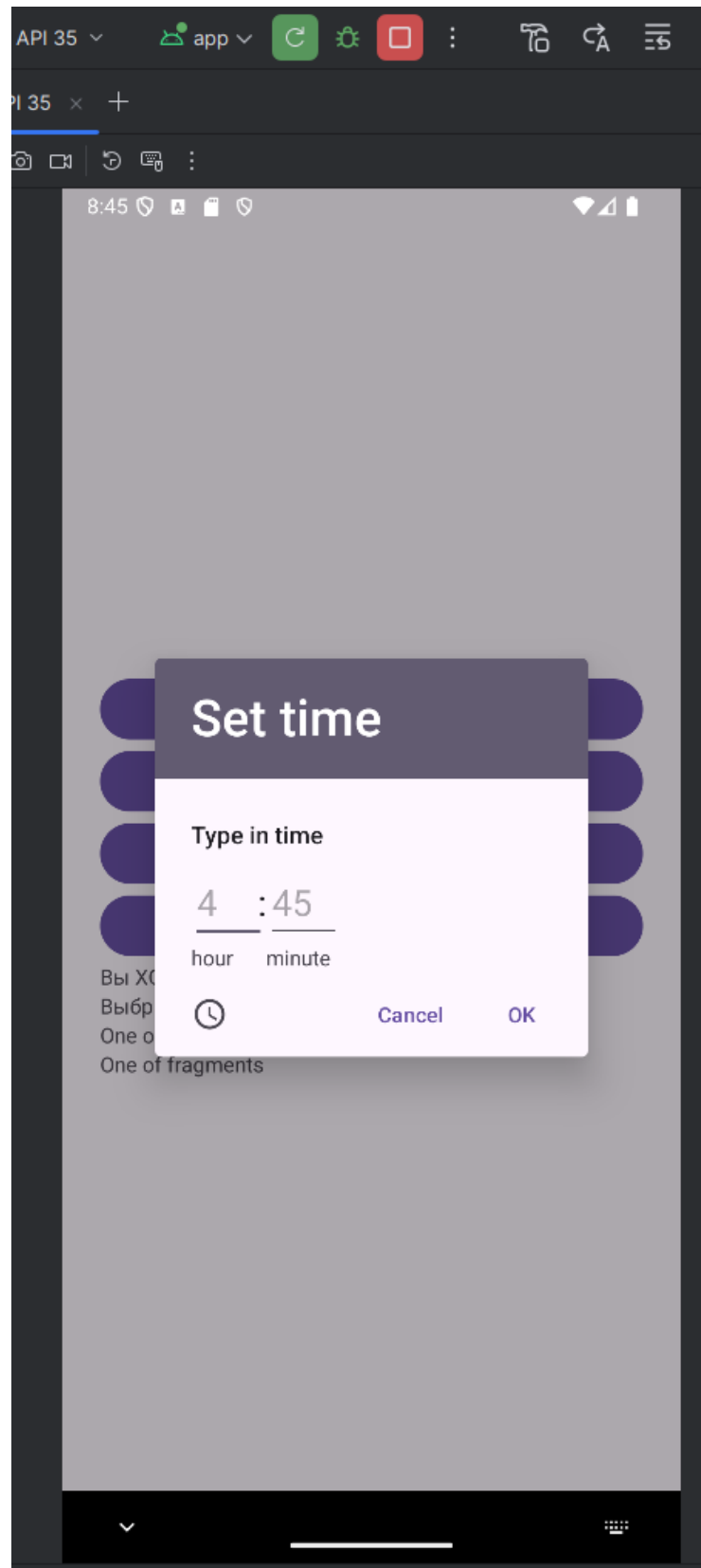


Рисунок 2.2.5.6 – Диалоговое окно класса TimePickerDialog

На рисунке 2.2.5.6 отображён выбор времени, предоставляемый пользователю при нажатии на кнопку с надписью TimePicker.

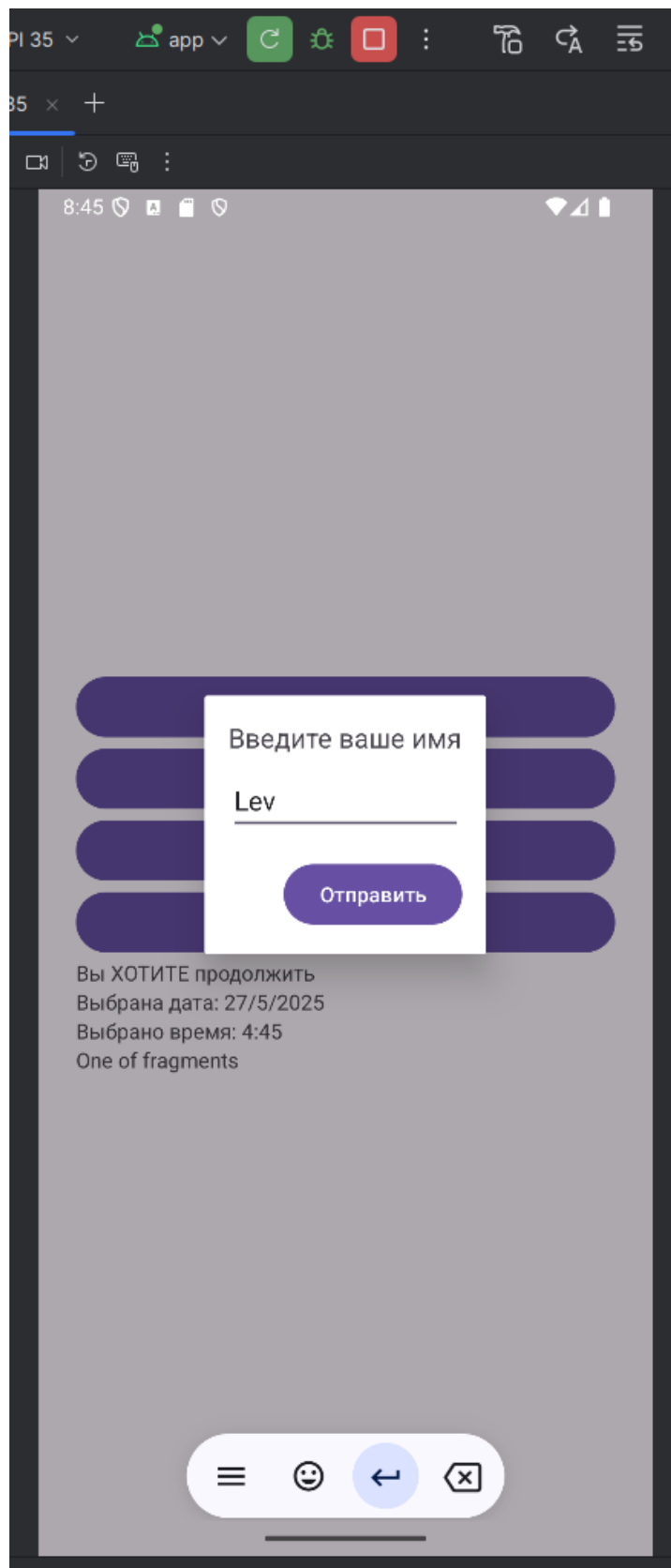


Рисунок 2.2.5.7 – Диалоговое окно класса Dialog

На рисунке 2.2.5.7 отображена возможность ввода текста при нажатии на кнопку с надписью Custom.

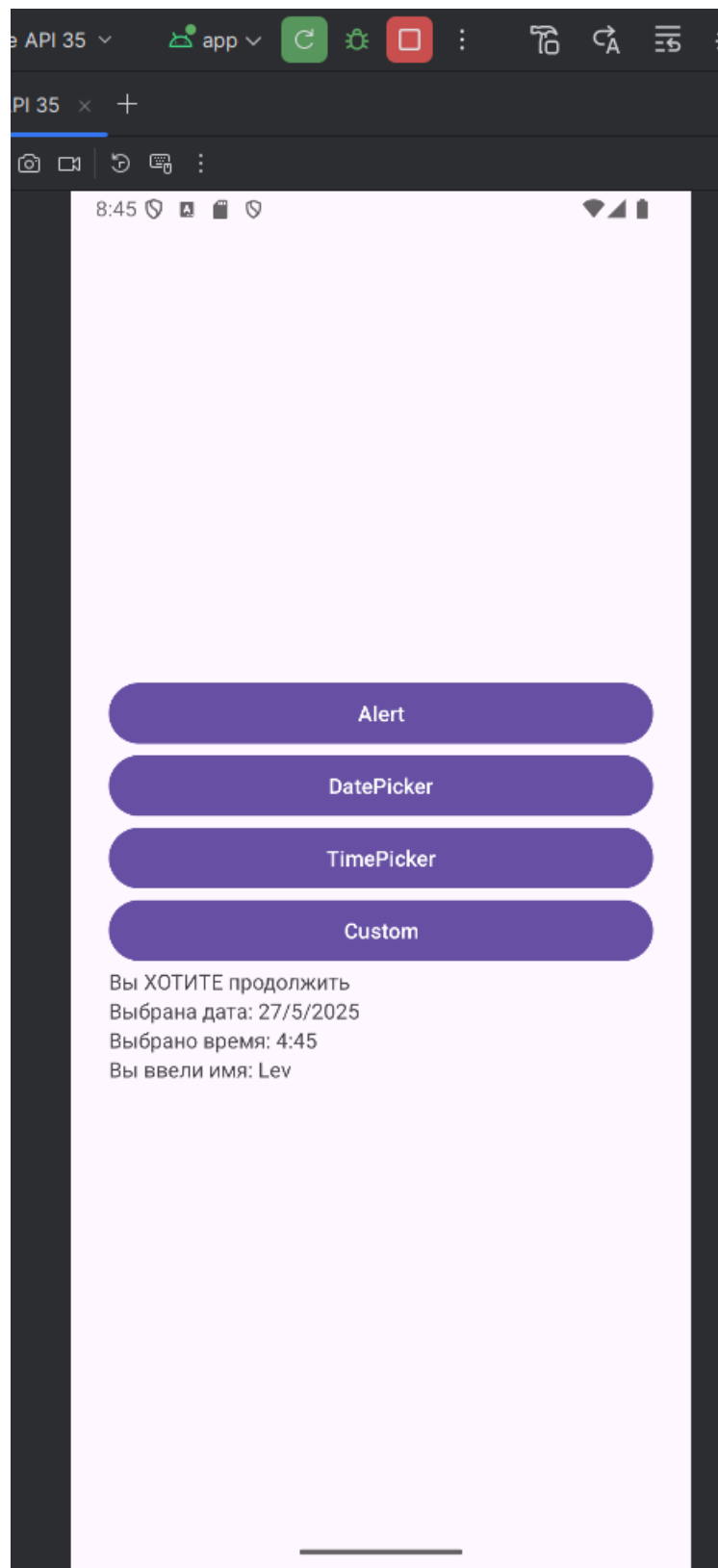


Рисунок 2.2.5.8 – Итоговый вид приложения

На рисунке 2.2.5.8 представлен итоговый вид приложения с отображением данных, полученных в результате взаимодействия пользователя с интерфейсом приложения.

ЗАКЛЮЧЕНИЕ

В ходе данной работы были получены знания в области взаимодействия с различными видами диалоговых окон в мобильных приложениях. Полученные знания были закреплены путём выполнения практического задания.